

```
In [1]: import pyreadr
import numpy as np
result = pyreadr.read_r("SMRi ordered - Brazil") # também Lê .RData
obj = next(iter(result.values())) # pega o único objeto do .rds

# se for um data frame R, vira um pandas.DataFrame
import pandas as pd
df = obj if isinstance(obj, pd.DataFrame) else None
df
```

```
Out[1]:
```

	C349_F	C509_F	C349_M	C509_M	C169_F	C61_M	C539_F	C159_F	C169_
0	1.123807	0.661231	0.986634	0.707814	1.221911	0.742698	1.529891	0.683831	1.3146
1	0.443441	0.610168	0.448988	1.158253	0.574836	0.669648	1.016424	0.642775	0.8570
2	0.879448	0.449045	0.890769	0.604989	1.166260	0.811681	0.525437	0.498478	0.8600
3	1.020218	0.422304	1.001927	0.662585	0.601234	1.225795	0.519147	0.668390	0.7399
4	0.443073	0.354335	0.906988	0.692725	0.575117	0.642738	0.642628	0.642295	0.6613
...	...	...	...	...	...	...	...	...	...
552	0.795859	0.878455	0.941837	0.501845	0.541944	0.668475	0.851317	0.613098	0.5979
553	0.736944	0.657660	0.301150	0.686377	0.532942	0.702059	0.835282	0.613446	0.6168
554	0.574846	0.596009	0.625308	0.505195	0.645011	0.514871	0.851198	0.707966	0.4493
555	1.210127	0.824201	0.832557	1.017706	0.747300	1.087854	1.233877	1.062437	0.5349
556	0.342973	0.419991	0.913603	1.234261	0.444595	0.794759	0.454757	0.277496	1.0593

557 rows × 30 columns



```
In [2]: vars_cols = df.var()
vars_cols.head()
```

```
Out[2]: C349_F    0.674739
C509_F    0.179864
C349_M    0.568566
C509_M    0.175342
C169_F    0.355013
dtype: float64
```

```
In [3]: df_std = df.copy()
X_real = df.copy()
#vamos padronizar as colunas
mu = df_std.mean(axis=0)
sd = df_std.std(axis=0, ddof=0) # desvio padrão populacional (ddof=0)

# evita divisão por zero (coluna constante). Aqui descartamos constantes.
keep = sd > 0
```

```
# Y é o DataFrame padronizado
Y = ((df_std.loc[:, keep] - mu[keep]) / sd[keep]).fillna(0.0)
print(Y.values, Y.shape)
Y.var().head()

[[ 0.36497084 -0.18883623  0.27358684 ...  0.17555823  1.919834
 -0.54308607]
 [-0.46404909 -0.30934677 -0.44008088 ... -0.18922218 -0.1473854
 -0.44313775]
 [ 0.06722115 -0.68960224  0.14633648 ...  0.05572086  0.17285073
 -0.75892984]
 ...
 [-0.30393368 -0.34276097 -0.20603454 ... -1.02813628 -0.87380289
 -0.46704425]
 [ 0.47015061  0.19577766  0.06906562 ... -0.29516127  0.58097297
 -0.02770818]
 [-0.58646782 -0.75816981  0.17664611 ... -0.04401604  1.72514341
 -0.75674525]] (557, 30)
```

```
Out[3]: C349_F    1.001799
        C509_F    1.001799
        C349_M    1.001799
        C509_M    1.001799
        C169_F    1.001799
        dtype: float64
```

```
In [10]: # Agora sim, podemos fazer a SVD
from numpy.linalg import svd

# X = M S Vt
#M, S, Vt = svd(Y.values, full_matrices=False)
M, S, Vt = svd(X_real.values, full_matrices=True)
print(M.shape, S.shape, Vt.shape) # M:(n×q), S:(q,), Vt:(q×q)

(557, 557) (30,) (30, 30)
```

```
In [5]: for i in S:
        print(f"{i:.3f}", end=" ")
```

```
125.860 26.175 20.187 16.404 15.537 13.614 12.404 12.043 11.446 10.524 9.268 8.895
8.612 8.134 7.858 7.274 7.159 6.743 6.663 6.425 6.164 5.960 5.853 5.547 5.334 4.919
4.806 4.522 4.416 3.404
```

```
In [9]: # Gavish-Donoho para escolher K
n, q = Y.shape

beta = min(q / n, n / q)
sigma_median = np.median(S)
omega = 0.56*beta**3 - 0.95*beta**2 + 1.82*beta + 1.43
tau = omega * sigma_median
k = np.sum(S >= tau)
print(f"n={n}, q={q}, beta={beta:.3f}, omega={omega:.3f}, tau={tau:.3f}, K={k}")
```

```
n=557, q=30, beta=0.054, omega=1.525, tau=20.351, K=6
```

The SVD approximation in (1) implies that the j th column $\mathbf{x}_j$ of the matrix $\mathbf{X}$, associated with the j th cancer, can be approximately written as

$$\begin{aligned}\mathbf{x}_j &\approx \sigma_1 \mathbf{v}_1[j] \times \mathbf{m}_1 + \cdots + \sigma_k \mathbf{v}_k[j] \times \mathbf{m}_k \\ &= \mathbf{W}_{1,j} \times \mathbf{m}_1 + \cdots + \mathbf{W}_{k,j} \times \mathbf{m}_k\end{aligned}\quad (3)$$

When the matrix $\mathbf{S}$ is dominated by a few large elements while the rest are relatively smaller, then it is possible to provide a good approximation to the data matrix $\mathbf{X}$ considering just part of the matrices $\mathbf{M}$ and $\mathbf{W}$. Defining k as the amount of significant values in the $\mathbf{S}$ matrix, $\mathbf{M}_k$ can be written as the $n \times k$ matrix containing the first k latent maps and $\mathbf{W}_k$ of dimension $k \times q$

being obtained selecting the first k rows of the matrix $\mathbf{W}$. Thus, $\mathbf{X}$ is approximated by:

$$\mathbf{X} \approx \mathbf{M}_k \mathbf{W}_k \quad (4)$$

With this, the n -dimensional column-vector of the j th cancer is given approximately by:

$$\mathbf{x}_j \approx \mathbf{M}_k \mathbf{W}_k[:, j]$$

where $\mathbf{W}_k[:, j]$ is the j th column of $\mathbf{W}_k$.

```
In [11]: import numpy as np

def xj_via_sum(M, S, Vt, j, k):
    """
    Implementa  $\mathbf{x}_j \approx \sum_{l=1..k} \sigma_l * \mathbf{v}_l[j] * \mathbf{m}_l$ 
    M: (n x q)    -> colunas  $\mathbf{m}_l$ 
    S: (q,)       -> valores singulares
    Vt: (q x q)  ->  $\mathbf{V}^T$ 
    j: índice 1-based da coluna
    k: posto da aproximação
    """
    x = np.zeros(n, dtype=M.dtype)

    for l in range(k):
        coeff = S[l] * Vt[l, j-1] #  $\sigma_l * \mathbf{v}_l[j]$ 
        x += coeff * M[:, l]
    return x

def xj_via_mat(M, S, Vt, j, k):
    """
    Forma matricial:  $\mathbf{x}_j \approx \mathbf{M}_k @ \mathbf{W}_k[:, j]$ , com  $\mathbf{W} = \mathbf{S} \mathbf{V}^T$ .
```

```

"""
Mk = M[:, :k] # (n x k)
Wk_col_j = S[:k] * Vt[:, k, j-1] # (k,) == [sigma_l * v_l[j]]_{l=1..k}
return Mk @ Wk_col_j

k=6
cols = [1, 10, 20, 30] # colunas pedidas

for j in cols:
    x_sum = xj_via_sum(M, S, Vt, j, k)
    x_mat = xj_via_mat(M, S, Vt, j, k)
    assert np.allclose(x_sum, x_mat)
    print(f"x_{j} via sum: {x_sum[:5]} ...")
    print(f"x_{j} via mat: {x_mat[:5]} ...")

```

```

x_1 via sum: [0.96123676 0.6274674 0.738298 1.01186298 0.78842609] ...
x_1 via mat: [0.96123676 0.6274674 0.738298 1.01186298 0.78842609] ...
x_10 via sum: [0.79577204 0.75937733 0.7077045 0.7201177 0.62722209] ...
x_10 via mat: [0.79577204 0.75937733 0.7077045 0.7201177 0.62722209] ...
x_20 via sum: [1.17586304 0.78303698 0.76091604 0.74670925 0.86386198] ...
x_20 via mat: [1.17586304 0.78303698 0.76091604 0.74670925 0.86386198] ...
x_30 via sum: [0.27118417 0.49297658 0.75349561 0.4666798 0.63228694] ...
x_30 via mat: [0.27118417 0.49297658 0.75349561 0.4666798 0.63228694] ...

```

In [12]: *#os X_j dos dados são basicamente quando k vale o total de valores singulares, ou s
#vamos calcular os X_j para k = 30 e comparar com os X^_j aproximados.*

```

k_full = len(S) # posto efetivo retornado pela SVD (<= min(n,q)) #30
k_approx = 6
cols = [1, 10, 20, 30] # 1-based como no artigo
for j in cols:
    x_approx = xj_via_mat(M,S,Vt,j,k_approx)
    x_exact = xj_via_mat(M,S,Vt,j,k_full)
    print(f"x_{j} approx (k=6): {x_approx[:5]} ...")
    print(f"x_{j} exact (k=30): {x_exact[:5]} ...")

data_col = X_real.iloc[:, j-1].to_numpy()
# verificação do "x_exato" com a coluna dos dados reais #tem que retornar True.
data_match = np.allclose(x_exact, data_col)
print(f"exact equals data column? {data_match}\n")

```

```

x_1 approx (k=6): [0.96123676 0.6274674 0.738298 1.01186298 0.78842609] ...
x_1 exact (k=30): [1.12380704 0.44344088 0.87944764 1.020218 0.44307269] ...
exact equals data column? True

x_10 approx (k=6): [0.79577204 0.75937733 0.7077045 0.7201177 0.62722209] ...
x_10 exact (k=30): [0.48397303 1.13993628 0.45852357 0.38568205 0.71454851] ...
exact equals data column? True

x_20 approx (k=6): [1.17586304 0.78303698 0.76091604 0.74670925 0.86386198] ...
x_20 exact (k=30): [1.02673339 0.95599176 0.95120745 0.69895393 1.33439582] ...
exact equals data column? True

x_30 approx (k=6): [0.27118417 0.49297658 0.75349561 0.4666798 0.63228694] ...
x_30 exact (k=30): [0.42142383 0.48856511 0.27642864 0.29695014 0.47260819] ...
exact equals data column? True

```

In [13]: `import matplotlib.pyplot as plt`

```

k_full = len(S) # posto efetivo retornado pela SVD (<= min(n,q)) #30
k_list = [3, 6, 10, 15]

# dicionários para guardar correlações
per_col_r = {k: {} for k in k_list}
r_global = {}

cols = np.arange(1, len(S)+1).tolist()

def build_pairs_and_corr(k):
    xs_all, ys_all = [], []
    per_col_r = {}
    for j in cols:
        x_approx = xj_via_mat(M, S, Vt, j, k)
        x_exact = xj_via_mat(M, S, Vt, j, k_full) # "exato" com k_full
        xs_all.append(x_exact)
        ys_all.append(x_approx)
        per_col_r[j] = np.corrcoef(x_exact, x_approx)[0, 1]
    xs_all = np.concatenate(xs_all)
    ys_all = np.concatenate(ys_all)
    r_all = np.corrcoef(xs_all, ys_all)[0, 1]
    return xs_all, ys_all, r_all, per_col_r

# --- calcula tudo primeiro para poder fixar os mesmos limites em todos os painéis
results = {} # k -> dict com xs_all, ys_all, r_all, per_col_r
all_xmins, all_xmaxs, all_ymins, all_ymaxs = [], [], [], []

for k in k_list:
    xs_all, ys_all, r_all, per_col_r = build_pairs_and_corr(k)
    results[k] = {"xs": xs_all, "ys": ys_all, "r": r_all, "per_col_r": per_col_r}
    all_xmins.append(xs_all.min()); all_xmaxs.append(xs_all.max())
    all_ymins.append(ys_all.min()); all_ymaxs.append(ys_all.max())

# mesmos limites para todos os gráficos
vmin = float(min(min(all_xmins), min(all_ymins)))
vmax = float(max(max(all_xmaxs), max(all_ymaxs)))

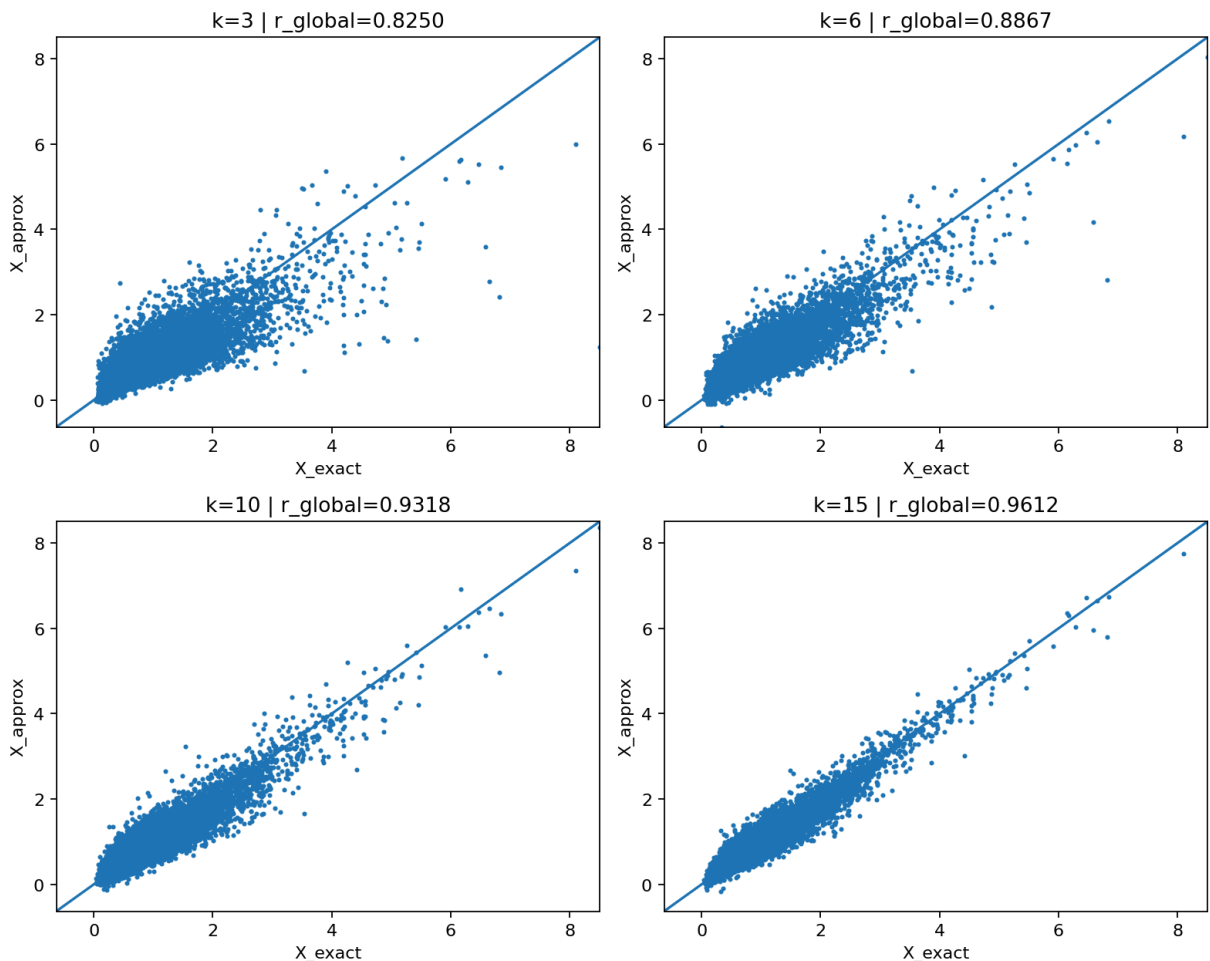
```

```
# --- subplots lado a lado (1x4)
fig, axes = plt.subplots(2,2, figsize=(10, 8), dpi=160)
axes = axes.ravel()

for ax, k in zip(axes, k_list):
    xs_all = results[k]["xs"]
    ys_all = results[k]["ys"]
    r_all = results[k]["r"]

    ax.scatter(xs_all, ys_all, s=3)
    ax.plot([vmin, vmax], [vmin, vmax]) # Linha y=x
    ax.set_xlim(vmin, vmax)
    ax.set_ylim(vmin, vmax)
    ax.set_xlabel("X_exact")
    ax.set_ylabel("X_approx")
    ax.set_title(f"k={k} | r_global={r_all:.4f}")

plt.tight_layout()
plt.show()
```



```
In [14]: print("Correlação por coluna (Pearson):")
for k in k_list:
    for j in cols:
        print(f"k={k:2d}  j={j:2d}  r={per_col_r[j]:.4f}")
```

Correlação por coluna (Pearson):

k= 3 j= 1 r=0.9880
k= 3 j= 2 r=0.9159
k= 3 j= 3 r=0.9806
k= 3 j= 4 r=0.9144
k= 3 j= 5 r=0.9244
k= 3 j= 6 r=0.8881
k= 3 j= 7 r=0.9232
k= 3 j= 8 r=0.9827
k= 3 j= 9 r=0.9476
k= 3 j=10 r=0.9735
k= 3 j=11 r=0.9674
k= 3 j=12 r=0.9500
k= 3 j=13 r=0.9483
k= 3 j=14 r=0.9714
k= 3 j=15 r=0.8356
k= 3 j=16 r=0.9551
k= 3 j=17 r=0.9120
k= 3 j=18 r=0.9564
k= 3 j=19 r=0.9411
k= 3 j=20 r=0.8556
k= 3 j=21 r=0.9775
k= 3 j=22 r=0.9935
k= 3 j=23 r=0.9934
k= 3 j=24 r=0.9985
k= 3 j=25 r=0.7394
k= 3 j=26 r=0.9650
k= 3 j=27 r=0.9992
k= 3 j=28 r=0.9404
k= 3 j=29 r=0.8520
k= 3 j=30 r=0.9622
k= 6 j= 1 r=0.9880
k= 6 j= 2 r=0.9159
k= 6 j= 3 r=0.9806
k= 6 j= 4 r=0.9144
k= 6 j= 5 r=0.9244
k= 6 j= 6 r=0.8881
k= 6 j= 7 r=0.9232
k= 6 j= 8 r=0.9827
k= 6 j= 9 r=0.9476
k= 6 j=10 r=0.9735
k= 6 j=11 r=0.9674
k= 6 j=12 r=0.9500
k= 6 j=13 r=0.9483
k= 6 j=14 r=0.9714
k= 6 j=15 r=0.8356
k= 6 j=16 r=0.9551
k= 6 j=17 r=0.9120
k= 6 j=18 r=0.9564
k= 6 j=19 r=0.9411
k= 6 j=20 r=0.8556
k= 6 j=21 r=0.9775
k= 6 j=22 r=0.9935
k= 6 j=23 r=0.9934
k= 6 j=24 r=0.9985
k= 6 j=25 r=0.7394

k= 6	j=26	r=0.9650
k= 6	j=27	r=0.9992
k= 6	j=28	r=0.9404
k= 6	j=29	r=0.8520
k= 6	j=30	r=0.9622
k=10	j= 1	r=0.9880
k=10	j= 2	r=0.9159
k=10	j= 3	r=0.9806
k=10	j= 4	r=0.9144
k=10	j= 5	r=0.9244
k=10	j= 6	r=0.8881
k=10	j= 7	r=0.9232
k=10	j= 8	r=0.9827
k=10	j= 9	r=0.9476
k=10	j=10	r=0.9735
k=10	j=11	r=0.9674
k=10	j=12	r=0.9500
k=10	j=13	r=0.9483
k=10	j=14	r=0.9714
k=10	j=15	r=0.8356
k=10	j=16	r=0.9551
k=10	j=17	r=0.9120
k=10	j=18	r=0.9564
k=10	j=19	r=0.9411
k=10	j=20	r=0.8556
k=10	j=21	r=0.9775
k=10	j=22	r=0.9935
k=10	j=23	r=0.9934
k=10	j=24	r=0.9985
k=10	j=25	r=0.7394
k=10	j=26	r=0.9650
k=10	j=27	r=0.9992
k=10	j=28	r=0.9404
k=10	j=29	r=0.8520
k=10	j=30	r=0.9622
k=15	j= 1	r=0.9880
k=15	j= 2	r=0.9159
k=15	j= 3	r=0.9806
k=15	j= 4	r=0.9144
k=15	j= 5	r=0.9244
k=15	j= 6	r=0.8881
k=15	j= 7	r=0.9232
k=15	j= 8	r=0.9827
k=15	j= 9	r=0.9476
k=15	j=10	r=0.9735
k=15	j=11	r=0.9674
k=15	j=12	r=0.9500
k=15	j=13	r=0.9483
k=15	j=14	r=0.9714
k=15	j=15	r=0.8356
k=15	j=16	r=0.9551
k=15	j=17	r=0.9120
k=15	j=18	r=0.9564
k=15	j=19	r=0.9411
k=15	j=20	r=0.8556
k=15	j=21	r=0.9775


```

k=15  j=22  r=0.9935
k=15  j=23  r=0.9934
k=15  j=24  r=0.9985
k=15  j=25  r=0.7394
k=15  j=26  r=0.9650
k=15  j=27  r=0.9992
k=15  j=28  r=0.9404
k=15  j=29  r=0.8520
k=15  j=30  r=0.9622

```

In [15]: *#Funcao adaptada do R para grafico de imagem*

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# --- helper: plota heatmap em um eixo com eixo-x 1..nc e y 1..nr ---
def _plot_imagem_ax(ax, y, vmin, vmax, cores=1, title=""):
    # aceita DataFrame ou array
    if hasattr(y, "values"):
        y = y.values
    y = np.asarray(y)
    nr, nc = y.shape

    cmap = "gray" if cores == 0 else "bwr"

    # extent faz o mapeamento 1..nc e 1..nr direto (sem meio-pixel)
    im = ax.imshow(
        y,
        origin="lower",
        vmin=vmin, vmax=vmax,
        cmap=cmap,
        interpolation="nearest",
        resample=False,
        aspect="auto",
        extent=[1, nc, 1, nr], # <<< eixo 1-based garantido
    )

    # ticks igualmente espaçados (e deduplicados)
    xticks = np.linspace(1, nc, 7).round().astype(int)
    xticks = np.unique(xticks)
    yticks = np.linspace(1, nr, 12).round().astype(int)
    yticks = np.unique(yticks)

    ax.set_xticks(xticks)
    ax.set_yticks(yticks)

    # formata como inteiros (evita '1.0', etc.)
    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, pos: f"{int(v)}"))
    ax.yaxis.set_major_formatter(FuncFormatter(lambda v, pos: f"{int(v)}"))

    ax.tick_params(axis="x", labelrotation=0, labelsz=9)
    ax.tick_params(axis="y", labelsz=9)
    ax.set_xlabel("colunas")
    ax.set_ylabel("linhas")
    if title:

```

```

        ax.set_title(title)
    return im

# --- reconstrução ( $X_k = M_k @ \text{diag}(S_k) @ Vt_k$ ) ---
def reconstruir_X_approx(M, S, Vt, k):
    Mk = M[:, :k]
    Wk = S[:, k, None] * Vt[:, k, :]
    return Mk @ Wk

# ===== Uso: dois gráficos lado a lado =====
k_approx = 6

# matriz original (se for DataFrame)
X = X_real.values
X_approx = reconstruir_X_approx(M, S, Vt, k_approx)

# mesma escala de cor para comparação justa
gmin = float(min(X.min(), X_approx.min()))
gmax = float(max(X.max(), X_approx.max()))

fig, axes = plt.subplots(1, 2, figsize=(12, 6), dpi=160, constrained_layout=True)

_ = _plot_imagem_ax(axes[0], X, vmin=gmin, vmax=gmax, cores=1, title="X (Re
im = _plot_imagem_ax(axes[1], X_approx, vmin=gmin, vmax=gmax, cores=1, title=f"X_ap

# colorbar única
cbar = fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.9)
cbar.set_label("valor")

plt.show()

```

